

PowerShell DSC (Desired State Configuration)

Objectifs :

- Comprendre le concept d'Infrastructure-as-Code (IaC) dans l'écosystème Windows.
- Identifier et contrer la dérive de configuration (Configuration Drift).
- Maîtriser l'architecture du LCM (Local Configuration Manager).
- Rédiger et compiler une configuration déclarative (.MOF).

1. L'Infrastructure-as-Code (IaC) et la Dérive de Configuration

Historiquement, les administrateurs déploient des serveurs via des scripts séquentiels (PowerShell, Batch). Cette approche "impérative" (indiquer *comment* faire les choses) montre ses limites à grande échelle.

1.1 Le problème du script impératif

Si un script PowerShell installe un rôle (ex: IIS) et qu'il est exécuté deux fois, il générera souvent une erreur la seconde fois car le rôle est déjà présent. De plus, il ne vérifie pas si l'état du serveur a changé depuis sa dernière exécution.

1.2 La Dérive de Configuration (Configuration Drift)

Le cauchemar des architectes est la dérive : un serveur est déployé parfaitement sécurisé le jour 1. Au fil des mois, des administrateurs (ou des attaquants) font des modifications manuelles (désactivation du pare-feu, arrêt du service Antivirus, ouverture de ports). Le serveur s'éloigne de son état de sécurité initial sans que personne ne s'en rende compte.

1.3 La solution DSC (L'approche déclarative)

PowerShell DSC est un moteur de gestion de configuration. L'administrateur n'écrit plus un script pour faire une action, il rédige un fichier texte déclarant **l'état final souhaité** du serveur (ex: "Le service pare-feu DOIT être démarré", "Le port 445 DOIT être fermé"). C'est le moteur DSC qui se charge de vérifier l'état actuel et de le corriger si nécessaire.

2. Architecture de PowerShell DSC

Le fonctionnement de DSC repose sur un composant central, natif dans toutes les versions modernes de Windows Server : le **LCM (Local Configuration Manager)**.

2.1 Le rôle du LCM

Le LCM est le "moteur" de DSC. Il tourne en tâche de fond sur la machine cible (Ring 3 / Espace utilisateur).

Son rôle est de recevoir une configuration, de l'appliquer, et de vérifier périodiquement (ex: toutes les 15 minutes) si le serveur respecte toujours cette configuration.

2.2 Les modes de fonctionnement du LCM

Le LCM peut être configuré de plusieurs manières pour gérer la dérive :

- **ApplyOnly** : Applique la configuration une fois. Si quelqu'un modifie le serveur par la suite, le LCM ne fait rien.
- **ApplyAndMonitor** : Applique la configuration et surveille. En cas de dérive, il se contente d'écrire une alerte dans l'Observateur d'événements (Event ID).
- **ApplyAndAutoCorrect (Le mode Architecte)** : Applique et corrige activement. Si un pirate arrête le service Sysmon, le LCM s'en rend compte au prochain cycle et redémarre le service automatiquement, empêchant ainsi la compromission à long terme.

3. Les Modèles de Déploiement (Push vs Pull)

Comment les configurations atteignent-elles les serveurs cibles ?

3.1 Le modèle PUSH (Pousser)

L'administrateur compile la configuration depuis sa station de travail et "pousse" manuellement le fichier vers les serveurs cibles via le protocole WinRM. C'est idéal pour des tests ou des environnements de Lab, mais difficilement scalable pour 1000 serveurs.

3.2 Le modèle PULL (Tirer)

C'est le standard de production (Architecture centralisée).

Un serveur central (Pull Server, souvent IIS ou Azure Automation) héberge toutes les configurations. Les LCM des 1000 serveurs cibles sont configurés pour contacter ce serveur central (en HTTPS) à intervalle régulier, demander s'il y a une nouvelle version de leur configuration, la télécharger et l'appliquer.

Si un nouveau serveur est provisionné, il lui suffit de connaître l'URL du Pull Server pour se configurer tout seul (Auto-provisioning).

4. Syntaxe et Ressources DSC

Une configuration DSC ressemble à un script PowerShell, mais utilise des mots-clés spécifiques.

4.1 Les Blocs de base

- **Configuration** : Le bloc englobant qui définit le nom du projet.

- **Node** : Définit la ou les machines cibles (ex: Node "SRV-DC01").
- **Ressources** : Les "ouvriers" de DSC (ex: WindowsFeature, Service, Registry, File).

4.2 La Compilation (Le format MOF)

Un fichier DSC (extension .ps1) ne s'exécute pas directement sur la cible. Il doit être **compilé**.

L'exécution du script génère un fichier universel d'infrastructure au format **.MOF (Managed Object Format)**. C'est ce fichier natif compréhensible par le système WMI de Windows qui sera envoyé et digéré par le LCM de la machine cible.

4.3 Exemple d'application sécurité (Contrer la désactivation de l'UAC)

Grâce à DSC, un architecte peut figer la clé de registre de l'UAC (EnableLUA). Si un logiciel tiers ou un attaquant tente de désactiver l'UAC dans le registre, le module Registry de DSC va détecter la dérive de la valeur (0 au lieu de 1) et réécrire la valeur à 1 lors de son prochain cycle de vérification.